

Architectuur- en ontwikkelstrategie

Invaarcalculator™ v3.9.2

IT-architectuur, workflow en validatie-strategie op hoofdlijnen

Innovation Nestor

12 juni 2026

Inhoudsopgave

1	Inleiding	3
2	Architectuur in één pagina	3
2.1	Wat de calculator is	3
2.2	Twee varianten: basic en expert	3
2.3	Distributie	3
2.4	Componenten	4
3	Technologie-keuzes	4
3.1	TypeScript voor engine en UI	4
3.2	Zod voor schema-validatie	4
3.3	esbuild voor bundeling	4
3.4	Web Worker voor de M3/RDR-module	4
3.5	pdfLaTeX voor documenten	5
3.6	Geen externe dependencies in productie	5
3.7	Geen tracking, geen ads, geen cookies	5
4	Codebase-structuur	5
4.1	Scheiding engine en UI	6
5	Ontwikkel-workflow	7
5.1	Sessies als ontwikkelingseenheid	7
5.2	De handoff-discipline	7
5.3	Versie-discipline	7
6	Validatie-strategie	8
6.1	Test-suite	8
6.2	Golden-fixture	8
6.3	Behavior-freeze voor refactors	8
6.4	Parity basic-expert	8
6.5	Jsdome-handmatige checks	9
6.6	CLI-toets van de RDR-engine	9
7	Samenwerking met AI-assistenten	9

7.1	Wat AI-assistenten doen	9
7.2	Waarom dit werkt	9
7.3	Wat de mens doet	9
7.4	Wat dit niet is	10
8	Open-source en licentie	10
9	Wat dit document niet beschrijft	10

1 Inleiding

Dit document beschrijft op hoofdlijnen hoe de Invaarcalculator is opgebouwd en hoe hij wordt ontwikkeld. Het is geschreven voor lezers die de inhoudelijke onderwerpen (Modelspecificatie, Demografie-verantwoording, M3/RDR-specificatie) elders al kennen en willen begrijpen hoe het kale instrument er onder de motorkap uit ziet — welke technologieën gebruikt zijn, hoe nieuwe versies tot stand komen, hoe correctheid wordt geborgd, en hoe de samenwerking met AI-assistenten in de praktijk werkt.

Dit is een actualisatie van de versie van 16 mei 2026 (v3.0.158). De grootste verschuivingen sindsdien: de calculator bestaat nu in twee varianten (*basic* en *expert*) uit één codebase, de expert-variant bevat een Monte-Carlo-module voor de risicodelingsreserve (M3/RDR) die live in de browser rekent, en de sessie-overdracht en validatie zijn aanzienlijk verder geformaliseerd.

2 Architectuur in één pagina

2.1 Wat de calculator is

Twee zelfdragende HTML-bestanden, gebouwd uit één codebase: **basic** (ongeveer 0,4 MB) en **expert** (ongeveer 17 MB), draaiend in elke moderne browser. Geen server, geen database, geen backend. Alle berekeningen voltrekken zich in JavaScript in de browser van de gebruiker; er gaat geen invoer over het netwerk de deur uit.

In beide varianten zitten de AG2024-sterftafel en de leeftijdgebonden DNB-rentetermijnstructuur als JSON ingebed. De expert-variant bevat daarnaast een subset van 10.000 scenario's uit de DNB-scenarioset (P-set) plus de RDR-rekenworker — vandaar het grootteverschil. De Modelspecificatie en de Demografie-verantwoording zitten, anders dan in eerdere versies, *niet* meer als PDF in het HTML-bestand ingebed; de documentatie staat als LaTeX-bron en PDF in de repository.

2.2 Twee varianten: basic en expert

Sinds de variant-splitsing levert dezelfde codebase twee producten:

- **Basic** — gratis variant voor deelnemers. Een wizard (triage → persoonlijke gegevens → financieel → resultaat) zonder technische sliders; de rekenrente is een vaste, leeftijdgebonden DNB-RTS-waarde. Een AOW-poort wijst geboortedata af waarvoor de AOW-leeftijd nog niet bereikt is (een ander uitkeringsregime dat de motor niet waardeert). Het resultaatsscherm toont naast het hoofdresultaat een dekkingsgraad × projectierente-gevoeligheidsmatrix.
- **Expert** — variant voor bestuur, helpdesk en audit. Alle parameters instelbaar (discontovoet, dekkingsgraad, bijstort, projectierente), met daarbovenop het M3/RDR-paneel (uitkeringswaaier P10/P50/P90), een fonds-brede collectieve pool-weergave en aanvullende grafieken.

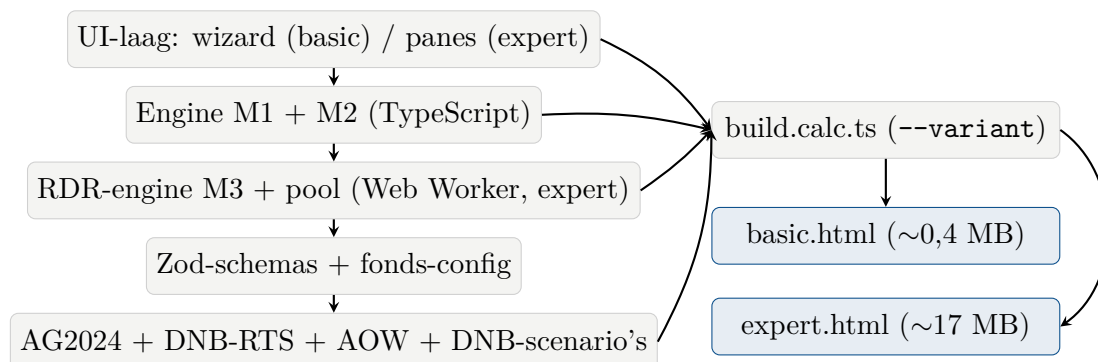
De splitsing is een build-as, geen tweede codebase. De build-flag `-variant=basic|expert` stuurt esbuild-aliassen (`~variant`, `~m2`, `~m3`, `~help`) die per variant óf de echte module óf een lege stub inbundelen. De basic-bundel bevat zo geen expert-bytes: geen M3-paneel, geen scenario-data, geen worker. Beide varianten hebben een eigen versieteller (zie §5.3); op het moment van schrijven basic 3.9.2 en expert 3.7.0.

2.3 Distributie

De broncode staat publiek op GitHub (github.com/Invaarcalculator/invaarcalculator). De build schrijft naast de versie-gestempelde bestanden ook `index.html` (kopie van basic) en `expert.html` (kopie van expert); GitHub Pages serveert die statisch op invaarcalculator.nl respectievelijk

invaarcalculator.nl/expert.html. Geen CI/CD-pipeline; geen automatische deployment; updates gebeuren door drag-and-drop van de bestanden in de GitHub-webinterface.

2.4 Componenten



Figuur 1: Component-overzicht. De engine-lagen voeren berekeningen uit; de UI-laag rendert resultaten; de schema-laag valideert de fonds-parameters; de bundler combineert alles per variant tot één bestand. De RDR-laag en de scenario-data gaan alleen mee in de expert-build.

3 Technologie-keuzes

3.1 TypeScript voor engine en UI

De gehele rekenkern, UI-rendering en build-tooling zijn in TypeScript geschreven. Reden: één taal voor alle code-paden, statisch type-systeem voor compile-time-fouten, en herbruikbaarheid tussen browser-runtime en server-side (Node.js) test-runner. De code is geschreven zonder framework: vanilla DOM-manipulatie, geen React/Vue/Svelte. Ook bij de huidige omvang (ongeveer 23.600 regels TypeScript, waarvan de UI-orchestratie in `src/ui.app.ts` alleen al ruim 4.200 regels) blijft die keuze houdbaar: de render-loop is één functie, state is één object, en er is geen build-afhankelijkheid van een framework-ecosysteem.

3.2 Zod voor schema-validatie

Alle fonds-parameters worden geladen via Zod-schemas. Dat geeft compile-time TypeScript-types en runtime-validatie van de invoer. Bij ongeldige fonds-configuratie faalt de calculator hard bij het laden, niet stilletjes bij een rekenstap. Het schema-bestand (`src/schema.v2.ts`) is ruim 1.100 regels lang en bevat per veld de bronvermelding en de toegestane waarde-range.

3.3 esbuild voor bundeling

De build-stap (`build.calc.ts`) gebruikt esbuild om alle TypeScript-bestanden in één JavaScript-bundel te compileren, voegt de embedded data toe (AG2024-sterftafel, DNB-RTS; voor expert ook de scenario-subset), en wraps alles in een HTML-template met inline CSS. De variant-keuze loopt via esbuild-aliassen (§2.2). Voor de expert-build draait een tweede esbuild-pass die de RDR-worker als zelfstandige IIFE-string bundelt; die string wordt in de HTML gebakken en pas in de browser via een Blob-URL tot Worker gemaakt — zo blijft de single-file-deploy intact, zonder los `.js`-bestand. Resultaat: per variant één zelfdragend HTML-artefact.

3.4 Web Worker voor de M3/RDR-module

Het M3-paneel (expert) draait een Monte-Carlo-projectie van de risicodelingsreserve live in de browser, in een Web Worker — buiten de UI-thread, zodat de pagina responsief blijft. De

waaier rekt live op 1.000 scenario's uit de embedded subset van 10.000 (visueel gelijkwaardige P10/P50/P90-banden tegen een fractie van de rektijd); herberekening gebeurt automatisch en gedebounded bij elke invoerwijziging. Dezelfde worker bedient ook de collectieve pool-modus (fonds-brede reserve-projectie); de berichten zijn getagd zodat waaier- en pool-resultaten elkaar niet kunnen verwarren.

3.5 pdfLaTeX voor documenten

De Modelspecificatie, de Demografie-verantwoording, de M3/RDR-specificatie en dit document zelf zijn pdfLaTeX-bronnen, gecompileerd naar PDF. Ze staan als plaintext-bron plus PDF in de repository; ze worden niet meer in het HTML-bestand ingebed. Reden voor LaTeX: typografische kwaliteit voor formele documentatie, en plaintext-bron die in versiebeheer kan zonder binary-diff-problemen.

3.6 Geen externe dependencies in productie

De gebouwde HTML-bestanden bevatten geen externe scripts, fonts, of API-aanroepen. Geen Google Fonts, geen jQuery-CDN, geen externe icoon-bibliotheek. Devtime-dependencies (TypeScript, Zod, jsdom voor tests, esbuild, tsx) zijn allemaal beperkt tot build- en test-fasen; in de gebruiker-zichtbare HTML zit alleen wat de gebruiker nodig heeft om de calculator te draaien.

3.7 Geen tracking, geen ads, geen cookies

De pagina zet geen cookies, gebruikt geen analytics, en stuurt geen invoer naar een server. De enige netwerk-activiteit is het éénmaal ophalen van het HTML-bestand bij paginaload via GitHub Pages. Wat GitHub Pages aan eigen logging doet (IP-adres, User-Agent, tijdstempel) staat buiten de invloed van deze tool en is gedocumenteerd in de privacy-sectie van de UI.

4 Codebase-structuur

Pad	Inhoud
<code>src/engine.m1.ts</code>	Kapitaalwaarde-berekening van de huidige aanspraak (KV), inclusief AG2024-sterftafel-lookups, annuïteitsfactoren, en joint-life-partner-mechaniek met broken-heart-correctie.
<code>src/engine.m2.ts</code>	Persoonlijk pensioenvermogen na invaren (PPV), inclusief fondsbalans-decompositie, cohort-aggregatie voor de fondsschalingsfactor x_{fonds} , en de alternatieve invaarmethode (Ortec-proxy).
<code>src/rdr/</code>	M3/RDR-module: <code>engine.rdr.ts</code> (individuele RDR-projectie, Monte-Carlo over de DNB-scenario's), <code>pool.rdr.ts</code> (collectieve fonds-brede reserve-pool), <code>rdr.worker.ts</code> (Web-Worker-entry), plus smoothing, scenario-subset-logica en de normatieve modulespecificatie <code>SPECIFICATIE_M3_RDR_v0.3.tex/.pdf</code> .
<code>src/schema.v2.ts</code>	Zod-schemas voor fonds-parameters en deelnemer-input; runtime-validatie met inline bronvermelding per veld.
<code>src/dnbrts.ts</code>	Leeftijdgebonden lookup op de embedded DNB-rentetermijnstructuur (curve in plaats van een vlakke scalar).
<code>src/belasting.ts</code>	Bruto/netto-conversie voor de netto-invoeroute in basic.

Pad	Inhoud
<code>src/ui.app.ts</code>	UI-orchestratie: state-management, event-handlers, modal-orchestratie, render-loop, de basic-wizard inclusief AOW-poort en resultaat scherm.
<code>src/ui.m2.ts</code> , <code>src/ui.m3.ts</code> (+ <code>.stub.ts</code>)	Expert-panes per rekenmodule (sliders, KPI-blokken, M3-waaier- en pool-pane). De <code>.stub.ts</code> -tegenhangers zijn lege drop-ins voor de basic-build.
<code>src/ui.modal.ts</code> , <code>src/ui.toast.ts</code> , <code>src/ui.shared.ts</code>	Accessibility-correcte modal-helper, niet-blokkerende toast-meldingen, gedeelde UI-hulpfuncties.
<code>src/variant.config.ts</code> (+ <code>variant.basic.ts</code> , <code>variant.expert.ts</code>)	Variant-definities: per-variant versieteller, feature-vlaggen, variant-specifieke teksten, plus de changelog per variant als commentaar.
<code>src/content/help/*.ts</code>	Tekstuele helpinhoud per parameter, gescheiden van rekencode; aparte registers voor basic en expert.
<code>fondsen/sspf/</code>	SSPF-specifieke parameter-waarden (fondsvermogen, dekkingsgraad, demografie), de Bijlage-B-ervaringssterftetabel en de worst-case-cohortmix voor de fondsmix-band.
<code>data/</code>	Externe bron-data: de DNB-RTS-CSV en het SVB-schema geboortemaand→AOW-leeftijd (basis van de AOW-poort).
<code>rdr_run/</code>	DNB-scenariosets en de herbouw-pijplijn voor de CLI-toets van de RDR-engine.
<code>fixtures/</code> <code>golden/sspf_default.json</code>	Vastgepinde anker-uitkomsten voor regressie-detectie.
<code>scripts/run-tests.ts</code>	Test-runner met manifest-gestuurde uitvoering en pass/fail/known-fail-categorisatie.
<code>scripts/snapshot.ts</code>	Sessie-afsluitings-tool: test, bouwt beide varianten, zippt de codebase, bundelt en verifieert de restart-set.
<code>build.calc.ts</code>	Bundle-script: combineert TypeScript-bundel en embedded data per variant in één HTML.

Totale codebase: ongeveer 23.600 regels TypeScript verdeeld over deze bestanden. De M1+M2-engine is ongeveer 85 KB; de RDR-laag (individuele engine + pool) komt daar met ongeveer 34 KB bovenop.

4.1 Scheiding engine en UI

Een bewuste discipline: rekencode en UI-code raken elkaar niet. De engine-bestanden (`engine.m1.ts`, `engine.m2.ts`, `engine.rdr.ts`, `pool.rdr.ts`) zijn pure functies — gegeven dezelfde input leveren ze dezelfde output, zonder DOM-afhankelijkheden, zonder side-effects. De UI-laag bouwt input-objecten op uit DOM-events, roept de engine aan, en rendert het resultaat; de RDR-worker is daartussen een dunne transportlaag. Deze scheiding maakt dat de engines los van een browser getest kunnen worden (engine-tier-tests in Node.js, plus een CLI-toets van de RDR-engine tegen dezelfde scenarioset als het paneel) en dat de UI-laag onafhankelijk gewijzigd kan worden (jsdom-tier-tests).

5 Ontwikkel-workflow

5.1 Sessies als ontwikkelingseenheid

Het werk gebeurt in afgeronde sessies van enkele uren tot enkele dagen. Een sessie begint met het uploaden van de *restart-set* uit de voorgaande sessie — vijf bestanden, ook gebundeld beschikbaar als `restart-set_v<X.Y.Z>.zip`:

- `Keep-14_v<X.Y.Z>_sessie.zip` — volledige werkdirectory-snapshot
- `00_START_HIER.md` — het startdocument: sessiegeschiedenis, openstaande punten en de vaste opstart- en afsluitprocedure
- `HANDOFF_v<X.Y.Z>.md` — wat er gewijzigd is, welke validatie gedaan is, openstaande items
- `calculator_v<X.Y.Z>_basic.html` en `calculator_v<X.Y.Z>_expert.html` — de gebouwde varianten voor visuele referentie

Een sessie eindigt met dezelfde set voor de nieuwe versie. De handoff is altijd leidend voor de actuele status; `00_START_HIER.md` houdt de keten van sessies vast en de zip bevat alle historische handoffs als referentie. Deze procedure bestaat omdat eerdere sessie-overgangen bij herhaling iets lieten vallen; het snapshot-script (§5.2) dwingt haar inmiddels machinaal af.

5.2 De handoff-discipline

Elke versie krijgt een eigen handoff-document met daarin:

- Wat is gewijzigd (welke bestanden, welke functies, welke gedragsverandering)
- Welke validatie is gedaan (test-resultaten, jsdom-verificatie, eventuele numerieke vergelijking)
- Open punten of vervolg-aanbevelingen
- Versie-bump-spoor: `src/variant.config.ts` (per-variant version), `package.json` version, en — bij engine-wijzigingen — `fixtures/golden_sspfd_default.json` `release_anchor`

Het snapshot-script faalt hard als de bijbehorende handoff ontbreekt, en verifieert na het bundelen dat de uitgekakte restart-set exact overeenkomt met de set die `00_START_HIER.md` voorschrijft — niet meer, niet minder. Dat is bewust: een versie zonder handoff is niet vindbaar bij toekomstige sessies, en zou de continuïteit van de samenwerking met AI-assistenten breken.

5.3 Versie-discipline

Soort wijziging	Versie-bump	Handoff-naam
Variant-specifiek (UI, teksten, resultaatscherm)	Alleen de geraakte variant	<code>HANDOFF_v<X.Y.Z>.md</code>
Gedeelde lagen (engine, fondsconfig, gedeelde content)	Beide varianten	<code>HANDOFF_v<X.Y.Z>.md</code>
Tooling (scripts, tests, docs)	Nee	<code>HANDOFF_v<X.Y.Z>-tooling.md</code>

Versie-nummers volgen het patroon `3.X.Y`: een minor-bump per inhoudelijke ronde, een patch-bump voor vervolgrondes binnen hetzelfde thema. (Tot en met v3.0.251 telde het patch-nummer simpelweg sessies; sinds v3.1.0 dragen minor-bumps betekenis.) Sinds de variant-splitsing heeft

elke variant een eigen teller in `variant.config.ts`, met de changelog als commentaar bij die teller. De tellers mogen divergeren: een reeks basic-only rondes laat expert staan — op het moment van schrijven basic 3.9.2 tegenover expert 3.7.0. Bij gedeelde wijzigingen bumpen beide. De codebase-zip en de restart-set dragen de hoogste van de twee.

6 Validatie-strategie

6.1 Test-suite

Bij elke sessie-afsluiting draait een volledige test-run (`npm run test:all`). Het manifest in `scripts/run-tests.ts` stuurt op dit moment 37 tests in twee tiers:

- **Engine-tier** (19 tests): pure TypeScript-tests draaiend in Node.js, geen browser nodig. Onder andere KV-smoke-tests, M2-cascade-tests, alternatieve-methode-consistentie, de M3-doorgifte- en grondslag-tests, de RDR-engine-, smoothing- en scenario-tests, een typecheck-guard, build-portability en de *golden-fixture-test* (zie hieronder).
- **Jsdom-tier** (18 tests): tests die de gebouwde HTML in een gesimuleerde browser laden en interactief gedrag verifiëren — end-to-end-berekening, komma-invoer, modal-toegankelijkheid, toast-gedrag, extensie-noise-filter, mobiele layout, typografie en dynamic type, de AOW-poort, de hoog/laag-volgorde, het parameter-scherm, en een parity-test tussen basic en expert.

Het manifest kent naast “pass” de categorie “known-fail” (bekend rood, telt niet als blokkade; een known-fail die *slaagt* wordt juist gemeld als kandidaat voor promotie). Op dit moment staat de gate op 37 “pass” en 0 “known-fail”. Een rode uitslag op een pass-test blokkeert de sessie-afsluiting: het snapshot-script weigert dan een restart-set te produceren.

6.2 Golden-fixture

Het bestand `fixtures/golden_sspf_default.json` bevat de exacte verwachte engine-uitkomst voor het UI-default-scenario (70-jarige man, EUR 100.000 pensioen, geen partner, SSPF-defaults). Bij elke engine-wijziging wordt deze fixture gecontroleerd: een afwijking van meer dan een fractie van een procent op KV, PPV of intermediate-grootheden vereist een expliciete update van het fixture-bestand mét bevestiging van de nieuwe “ankerwaarde”. Ook een bewuste data-update (zoals het verwerken van nieuwe jaarverslag-cijfers in de `fondsconfig`) loopt langs deze route: de fixture wordt dan herijkt en de herijking wordt in de handoff verantwoord.

Dit beschermt tegen drift: kleine code-wijzigingen die per ongeluk een numerieke output verschuiven, slaan stuk op de golden-fixture-test en moeten dan verantwoord worden in de handoff voordat de fixture wordt geactualiseerd.

6.3 Behavior-freeze voor refactors

Bij refactor-stappen (opschoning, herstructurering, dode-code-verwijdering) gebruiken we een zwaardere check: de gebundelde JavaScript-output wordt per variant byte-voor-byte vergeleken met de vorige versie. Identiek (op de versie-string na) = bewezen geen gedragsverandering. Een refactor die de bundel-bytes onveranderd laat, is per definitie gedrag-neutraal en kan zonder verdere numerieke verificatie aanvaard worden.

6.4 Parity basic–expert

De engines van basic en expert zijn identiek; de varianten verschillen in UI en in default-keuzes (basic rekent op de vaste leeftijdgebonden DNB-RTS-waarde, expert laat de discontovoet instellen). Een geautomatiseerde parity-test in de jsdom-tier bewaakt dat beide builds bij gelijke invoer

dezelfde kernuitkomsten geven, zodat de variant-splitsing een presentatie-laag blijft en geen rekenverschil kan worden. Daarnaast wordt bij variant-specifieke rondes via jsdom geverifieerd dat de niet-geraakte variant ook werkelijk ongewijzigde zichtbare output heeft.

6.5 Jsdom-handmatige checks

Voor UI-wijzigingen die de normale test-suite niet exhaustief dekt (nieuwe knoppen, modal-volgorde, dropdown-gedrag, het resultaatscherm), schrijven we per sessie ad-hoc jsdom-scripts die de gebouwde HTML laden en de specifieke wijziging verifiëren. Deze scripts staan niet in versie-controle — ze zijn wegwerp-instrumenten voor éénmalige verificatie en worden in de handoff genoteerd onder “Verificatie”. De keuze wat wél een geregistreerde test wordt, is bewust terughoudend: tests pinnen gedrag vast, en teksten of layout die nog itereren met de auteur worden niet voortijdig vastgepind.

6.6 CLI-toets van de RDR-engine

De RDR-engine kan buiten de browser worden gedraaid op de volledige DNB-scenarioset (pijplijn in `rdr_run/`). Het M3-paneel is bij oplevering geverifieerd tegen die CLI-route (parity tussen paneel-uitkomsten en CLI-uitkomsten op dezelfde scenario-subset), zodat de browser-integratie — worker, subset, doorwerking $M1 \rightarrow M2 \rightarrow M3$ — geen eigen rekenwerkelijkheid kan ontwikkelen.

7 Samenwerking met AI-assistenten

7.1 Wat AI-assistenten doen

De codebase is iteratief tot stand gekomen in dialoog met meerdere AI-assistenten: voornamelijk Anthropic Claude (diverse versies, op het moment van schrijven Claude Fable 5) voor TypeScript-implementatie, modelspecificatie-redactie en architectuurkeuzes; PensioenGPT voor domeinspecifieke navigatie van Pensioenwet en bijlagen; aanvullende assistenten voor kruiscontrole en onderzoeksvragen.

7.2 Waarom dit werkt

Drie ontwerpkeuzes maken de samenwerking met AI-assistenten praktisch beheersbaar:

- **Pure functies in de engine.** Engine-code zonder side-effects is testbaar en reviewbaar zonder uitvoer-context; een AI-assistent kan een functie lezen, een test schrijven en de uitkomst beredeneren, allemaal op tekst-niveau.
- **Zware schema-validatie.** Zod-fouten zijn expliciet en bevatten de context (welk veld, welke regel, welke waarde). Een AI-assistent die per ongeluk een verkeerd type-veld opstelt, krijgt direct een leesbare foutmelding terug die in de volgende iteratie de fix oplevert.
- **Handoff-conventie.** Elke sessie sluit af met een handoff die de volgende sessie heropent zonder context-verlies, met `00_START_HIER.md` als vaste ingang. De handoff is leesbaar voor zowel mens als AI-assistent — ze beschrijft niet alleen *wat* gewijzigd is maar ook *waarom*, zodat de volgende iteratie niet opnieuw door dezelfde argumenten hoeft.

7.3 Wat de mens doet

De rolverdeling in een sessie: de gebruiker (auteur) bepaalt richting en doel — welk probleem, welke prioriteit, welke compromissen acceptabel zijn. De AI-assistent implementeert, test en verifieert, met de gebruiker als reviewer en eindbeslissers. Bij elke substantiële keuze (architectuur,

terminologie, modelaannamen) levert de AI-assistent meerdere opties met afwegingen; de gebruiker selecteert.

7.4 Wat dit niet is

De AI-assistenten zijn geen autonomous-agents die zelf code committen. Elke wijziging gaat door dezelfde sessie-cyclus: handoff lezen, wijziging voorstellen, code maken, tests draaien, verificatie tonen, handoff schrijven, snapshot leveren. De gebruiker downloadt de restart-set en kiest of hij publiceert door `index.html` en `expert.html` in GitHub te plaatsen.

Aan de tool kunnen geen rechten worden ontleend; de inhoudelijke verantwoordelijkheid voor wat erin staat ligt bij de auteur, niet bij de AI-assistenten. Aanwezigheid van fouten — in interpretatie van de Pensioenwet, in actuariële formules, in code, of in numerieke uitkomsten — kan niet worden uitgesloten en is op een aantal punten statistisch waarschijnlijk.

8 Open-source en licentie

De volledige broncode is publiek beschikbaar onder CC BY-NC 4.0 (Creative Commons Naamsvermelding-NietCommercieel 4.0 Internationaal). Dat betekent dat onafhankelijke verificatie van de berekeningen mogelijk is, dat afgeleide werken zijn toegestaan met bronvermelding, en dat commercieel gebruik expliciete toestemming vereist.

De documentatie (Modelspecificatie, Demografie-verantwoording, M3/RDR-specificatie, dit document) valt onder dezelfde licentie. Alle bronnen zijn als plaintext LaTeX in de codebase aanwezig zodat correcties zichtbaar zijn in versie-controle en niet in een binary-format zitten.

9 Wat dit document niet beschrijft

- **De rekenformules zelf.** Die staan in de Modelspecificatie; de M3/RDR-module heeft een eigen normatieve specificatie (`SPECIFICATIE_M3_RDR_v0.3` in `src/rdr/`). Dit document beschrijft alleen de *infrastructuur* eromheen.
- **De cohort-data.** Bron, aging-methodiek en M/V-uitsplitsing van de SSPF-cohortbezetting staan in de Demografie-verantwoording.
- **Specifieke implementatie-details.** Voor architectuur op functie-niveau: zie de inline-commentaar in de TypeScript-bestanden zelf en het bestand `ENGINE_SPECIFICATIE_v1.md` in de codebase (normatief specificatie-document, niet primair gebruikersgericht).
- **Roadmap of geplande wijzigingen.** Per-sessie-status staat in de meest recente handoff en in `00_START_HIER.md`; een centrale roadmap-tracker bestaat niet.

Einde architectuur- en ontwikkelstrategie.